OpenZeppelin | security

Sui Token Pre-Sale Audit

July 17, 2026

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

Prefunded Sale 5

Pricing Curve 6

Refund Vault 6

Security Model and Trust Assumptions 7

Low Severity 9

L-01 Integer-Only Rate in mint_quote Constrains Representable Prices 9

L-02 Unenforced Coherence Between Vesting Type Parameters Can Permanently Strand Allocations 10

L-03 Documentation Invites a Publicly Constructible Vesting Witness That Voids the Schedule Lockup 11

L-04 Unvalidated Sale Window Can Permanently Freeze Funds on an Allowlist-Gated Sale 12

Notes & Additional Information 13

N-01 Event Schemas Omit Enforcing Types, Hindering Event-Only Filtering 13

N-02 Misleading Error Messages 14

N-03 Quote Does Not Guarantee Fresh Sale State Within a Transaction 15

N-04 Vesting Lockup Guarantee Is Not Implied by the Documented Curve Contract 16

N-05 Missing Documentation 17

N-06 Code Style Inconsistencies 18

N-07 External Integrators Cannot Reliably Distinguish Finalized From Cancelled Sales 19

N-08 Per-Buyer Cap Accounting Documented as an Accumulating Total 20

N-09 Incorrect Documentation 21

N-10 Load-Bearing ENotInit Guard Mislabeled as Purely Defensive 23

N-11 Inconsistent Zero-Amount Event Suppression 24

N-12 Lifecycle Events Are Not Self-Contained for Off-Chain Consumers 25

N-13 Missing Batch Refund Counterpart to claim_all 26

Conclusion 28

Appendix 30

Issue Classification 30

Summary

Type	Library
Timeline	From 2026-07-10 To 2026-07-16
Languages	Move

Total Issues	17 (0 resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	0 (0 resolved)
Low Severity Issues	4 (0 resolved)
Notes & Additional Information	13 (0 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the OpenZeppelin/contracts-sui repository at commit 6fcf34c.

In scope were the following files:

```
contracts
    └── sale
        └── sources
            ├── allowlist.move
            ├── fixed_rate_curve.move
            ├── prefunded_sale.move
            ├── receipt.move
            └── refund_vault.move
```

System Overview

The `openzeppelin_sale` package implements a prefunded token sale. An issuer deposits the entire sale inventory up front, fixes a pricing rule, and opens the sale to buyers for a bounded time window. Payments are collected as buyers purchase, and the outcome is decided by two thresholds: a hard cap bounds the total raise, and an optional soft cap sets the minimum below which the sale is considered failed. A sale that closes at or above its soft cap is finalized, letting buyers redeem their purchased tokens and the issuer withdraw the proceeds. A sale that falls short is cancelled, moving the collected payments into an escrow from which buyers reclaim them.

The package centers on a shared `PrefundedSale` object that holds the inventory, the collected proceeds, and the sale's configuration and lifecycle state. Pricing is delegated to a separate curve module, so the sale mechanism is independent of any particular pricing rule; the package ships one fixed-rate curve. A paired `RefundVault` object escrows the proceeds on cancellation. Two supporting modules complete the package: an allowlist for compliance-gated sales, and a receipt type that records each buyer's purchase. All configuration is fixed during an initialization phase and becomes immutable once the sale is activated and shared.

The engagement covered the five modules under `contracts/sale/sources`. A sale configured with a vesting schedule releases tokens into a vesting wallet from the separate `openzeppelin_finance` package, which was audited separately and is out of scope for this audit.

Prefunded Sale

The `prefunded_sale` module is the stateful core that orchestrates the lifecycle. A sale moves through its phases in order: `Init`, then `Active`, then one of the two terminal phases, `Finalized` or `Cancelled`. During `Init` the sale is an owned value held by its creator, so possession is the only authority its configuration needs; activation shares the object, after which its configuration can no longer change.

- Creation and funding: `create_sale` returns the owned sale, and `deposit` adds the sale-token inventory.
- Configuration, each `Init`-only and one-shot: a per-buyer cap through `set_per_buyer_cap`, the refund vault through `pair_refund_vault`, a compliance

allowlist through `enable_allowlist`, and an optional vesting schedule through `set_vesting_schedule_params` that redirects redemption into a time-locked wallet.
- Activation: `share_and_activate` checks the deposited inventory against the backing the curve commits to, shares the sale and its vault together, and moves the sale to `Active`.
- Purchasing: a buyer drives `purchase` with a quote minted by the curve.
- Closing: `finalize` on a successful sale, `cancel_after_close` on one that closed below its soft cap, or the admin-only `cancel_emergency`.
- Redemption: `claim` and `claim_all` return purchased tokens after a finalize, `refund` returns payments after a cancel, and `withdraw_proceeds` and `withdraw_unsold_inventory` pay the issuer.

Pricing Curve

The `fixed_rate_curve` module is a stateless pricing rule that the sale consumes through a witness-gated interface. The sale does not price a purchase on its own; the curve mints two carriers that the sale accepts, each of which has no abilities and so must be produced and consumed within one transaction.

- `params` constructs the rate, and `activation_ticket` commits the inventory backing the sale must hold to activate.
- `quote` prices a payment and returns a quote carrying that payment and the computed token allocation to `purchase`.

Refund Vault

The `refund_vault` module is a generic, cap-controlled escrow over a single coin type, usable on its own or paired with a sale. It holds a balance and a one-way lifecycle state, `Active` and then `Refunding` or `Closed`, and only the holder of its controller cap can move funds or advance that state.

- `new` creates an `Active` vault and returns its controller cap.
- `deposit` adds funds, and `flip_to_refunding` or `flip_to_closed` sets the terminal state.
- `release_balance` pays out a chosen amount while `Refunding`, and `withdraw_all` drains the vault while `Closed`.

Security Model and Trust Assumptions

The sale enforces its guarantees through Move's type system and a capability model rather than runtime access control. Purchased tokens and refunds are gated by non-transferable receipts bound to the purchasing account. The close is open to any caller and a refund needs only the buyer's own receipt, so neither depends on the issuer. Beyond that, the system's safety rests on the issuer configuring the sale sensibly, the pricing curve behaving honestly, and the integrity of the underlying Sui object, clock, and transfer model.

The system relies on the following trust assumptions:

- The price a buyer pays is set by the curve module, so an integrator adopting a custom curve trusts that curve for correct pricing.
- The soft cap bounds the amount raised, not its source. An issuer can meet it with their own capital, and by buying up to the hard cap can trigger an early `finalize` and recover that capital through `withdraw_proceeds` in the same transaction, at near-zero net cost. The soft cap therefore protects buyers against an underfunded sale, not against a determined issuer.
- Buyers must retain their receipts and redeem from the account that purchased. Receipts are non-transferable, and both `refund` and `claim` require the caller to be the recorded buyer.

The system defines the following privileged roles:

- **Issuer (`SaleAdminCap` holder):** configures the sale during `Init` and then activates it, through `create_sale`, `deposit`, `set_per_buyer_cap`, `pair_refund_vault`, `enable_allowlist`, `set_vesting_schedule_params`, and `share_and_activate`. After a terminal transition, the issuer withdraws the proceeds through `withdraw_proceeds` and any unsold inventory through `withdraw_unsold_inventory`, and alone can emergency-cancel a live sale through `cancel_emergency`.
- **Allowlist admin (`AllowlistAdmin` holder):** on a compliance-gated sale, mints the single-use entries that authorize individual purchases, through `new_entry`.
- **Curve module (witness holder):** the only party that can mint a quote for a sale of its curve type, since `mint_quote` consumes the curve witness by value.

- **Buyer:** drives the paths open to any holder of a valid receipt, or an allowlist entry where one is required: `purchase`, `claim`, `claim_all`, and `refund`. Both `finalize` and `cancel_after_close` are permissionless once the sale's window and cap conditions are met, so any caller can drive the close.

Low Severity

L-01 Integer-Only Rate in `mint_quote` Constrains Representable Prices

Pricing is not performed by the curve. `mint_quote` takes a `rate: u64` as a plain argument and computes the allocation itself, as a widened product of the payment value and that rate. The shipped `fixed_rate_curve::quote` merely reads its stored rate and passes it through. Since `Quote` declares module-private fields and `mint_quote` is its only constructor, no curve can mint a quote any other way. The form `allocation = paid * rate`, with an integer rate, therefore binds every curve that can be written against this sale. A curve may vary the rate per call, but only the multiplier varies, never the functional form.

The consequence is a hard ceiling on price. The rate is a whole number and cannot fall below 1, so the allocation is never smaller than the payment measured in base units, and the sale cannot price a token above the point where one payment base unit buys one sale base unit. For a token sold against a stablecoin of the same decimals, that ceiling is one payment token per sale token: the nearest configurable prices to 5.00 are 1.00 at `rate = 1` and 0.50 at `rate = 2`, and the whole range between is unreachable. This is not a rounding artifact but an entire region of price space that the sale cannot express. Below the ceiling only integer-divisor fractions of it are exact, so an off-grid target must be approximated by a nearby integer rate, mispricing every purchase. The rate's meaning is documented as sale tokens in smallest units per one payment-coin smallest unit, but neither the ceiling nor the grid is stated anywhere, so the mispricing is fixed silently at configuration time with no abort and no anomalous event.

No custom curve can escape this, which is where the extension seam fails to reach. A curve may divide when computing its rate, but the result truncates to a `u64` and the sale still applies `paid * rate`, so division never reaches the allocation, and an integrator needing an off-grid price on a given decimal pairing must fork the sale itself. The package also documents the opposite of what it implements: the design commentary states that the sale accepts a curve-computed allocation and applies it verbatim, with correct pricing delegated to the curve. The verbatim half is accurate, since `purchase` uses the carried allocation bounded only by the inventory check. The curve-computed half is not: the curve supplies a rate, and the sale computes the allocation. The documented design is therefore strictly more expressive than the implemented one, since a curve passing a computed allocation directly could do its own widened division and express any price on any decimal pairing.

Consider documenting the price envelope on `mint_quote` and on the curve's `params` constructor, stating the ceiling and the representable grid with a worked example. Consider also aligning the implementation with the design the package already describes, letting the curve supply a computed allocation directly rather than a rate, which is a breaking change to `mint_quote` and so affects every curve.

L-02 Unenforced Coherence Between Vesting Type Parameters Can Permanently Strand Allocations

The `PrefundedSale` struct pins `VestingWitness` and `VestingScheduleParams` as independent type parameters, and the struct documentation requires them to form a coherent pair, describing the witness as the schedule witness of the curve module that interprets the params. Nothing enforces that pairing. `set_vesting_schedule_params` accepts a params value and stores it, checking only the phase and that no schedule is already attached, and no function in the module ever takes a value of the witness type, so the witness is constrained by nothing at all. An issuer who names one curve's witness while supplying another curve's params therefore configures an incoherent pair, and the sale accepts it: the configuration compiles, activates, and sells. Mixing two curves is what the composition requires, and the vesting wallet documents a recipe for downstream packages to ship their own curve alongside the built-in linear one, so a second curve is anticipated by design.

`claim_into_vesting` and `claim_all_into_vesting` build and fund a wallet over that pair with the buyer's whole allocation, and it can never be released. Releasing requires an attestation from `mint_vested_amount`, which is gated on a witness value. Where the pinned witness is a curve's module-private type, as it is here, only the declaring module can construct one, and it mints only inside functions typed to its own params: the linear curve's `vested_amount` accepts a wallet over `Linear` and its own `Params`, and nothing else. An attestation minted from a coherent side wallet does type-check against the mismatched one, but `releasable` asserts that the attestation names the wallet it is presented against, so that rescue closes as well, and `destroy_empty` asserts an empty balance, closing teardown. The loss commits inside the claiming transaction, because `claim_internal` consumes the receipt and decrements the sale's counters before the wallet is built: nothing aborts, so the buyer's claim is destroyed in the same call that mints the unreleasable wallet, with no second attempt and no recovery path for any party. The concern is issuer error rather than an attacker, and the cost falls entirely on buyers, who have no influence over the type arguments.

Consider having the curve, rather than the issuer who cannot construct a module-private witness, supply the schedule parameters and the witness together as a single value that only the declaring module can build, so that the two slots unify against each other and an incoherent pair stops compiling. The cost is one constructor per curve and a breaking change to the sale's setter. Consider also stating in the struct documentation that the pairing is unchecked, so that an integrator shipping a second curve knows the two slots must be kept in step by hand.

L-03 Documentation Invites a Publicly Constructible Vesting Witness That Voids the Schedule Lockup

The sale documents its vesting lockup as unconditional. The `set_vesting_schedule_params` documentation asserts that "the schedule cannot be bypassed" and gives as its reason that "only the curve module that owns that witness can mint the `VestedAmount` needed to release". That reason holds only where the pinned witness is a curve's module-private type, and nothing requires it to be one: `mint_vested_amount` is bounded only by `S: drop`, which an ordinary public type such as `bool` satisfies.

Where the pinned witness is such a type, the value gating attestation is one any caller can construct, so the wallet's holder can mint an attestation for the wallet's entire total, which the public `balance` and `released` views expose, and release the whole balance on the first call regardless of the schedule. The sale's own documentation directs issuers into exactly that configuration, stating that for a non-vesting sale "both slots are inert - pick any `drop` witness and any `copy + drop + store` params type and never attach a schedule". The advice is safe only under that final clause, and nothing re-checks the slot when a schedule is later attached, so a sale built on a throwaway witness and then given a schedule ships with no lockup at all. The package README contradicts the advice and models the safe pattern, instructing that the witness slot be instantiated with `vesting_wallet_linear::Linear` even for a non-vesting sale. The bypass is not theft, since `release` pays the wallet's recorded beneficiary and the tokens reach the buyer either way; the cost falls on the schedule the issuer sold to the market, which any holder can void on day one.

Consider deleting the "pick any `drop` witness" advice and directing issuers to name a real curve's witness even where the slot is inert. Because no type bound can express that a witness's constructor is module-private, the remedy is documentation. A module-private witness is harmless in an unused slot, so naming one costs nothing and removes the hazard for any issuer following the documentation.

L-04 Unvalidated Sale Window Can Permanently Freeze Funds on an Allowlist-Gated Sale

The purchase window governs every exit from a live sale, and `create_sale`'s only validation of the window is that `opens_at_ms` precedes `closes_at_ms`. There is no maximum duration and no `Clock` parameter, so the function cannot compare the window against the present, and a close set beyond any reachable date is accepted. Separately, an allowlist-gated sale admits a purchase only against an `AllowEntry`, which only the holder of the `AllowlistAdmin<SaleCoin>` can mint. That capability can never be replaced, because `enable_allowlist` is restricted to the `Init` phase and runs at most once, and entries cannot be stockpiled against its loss, because `AllowEntry` is declared with no abilities and so cannot outlive the transaction that mints it.

These two properties compose into a fund freeze. On a sale whose nonzero soft cap is met while `raised` remains below `hard_cap`, `finalize` aborts because the window is open and the hard cap is unmet, `cancel_after_close` aborts because the window is open, and `cancel_emergency` aborts with `ESoftCapMet` even for the `SaleAdminCap` holder. That interval is deliberate, since those guards exist to prevent rugging a successful sale, and its intended terminator is the close of the window. The only exit that does not wait for the window is advancing `raised` to the hard cap so that `finalize` passes on its hard-cap disjunct, and that requires purchases. Losing the `AllowlistAdmin` therefore removes the last exit, and with a close that never arrives the sale is stranded in `Active` permanently: the payments in the sale's `proceeds` balance, the unallocated inventory behind `withdraw_unsold_inventory`, and every buyer's allocation behind claiming's `Finalized` guard are locked with no party able to release them. The window is the enabling condition rather than the key loss: on a sale that closes at a reachable date, the same lost capability strands nothing, because the window closes, the permissionless `finalize` succeeds on the met soft cap, and buyers claim. Yet footgun 3 describes the loss as costing only future purchases, and the `allowlist` module's own paragraph goes no further than calling the sale "uncompletable". Neither warns that a far-future close turns the same accident into a total loss of money already taken.

Consider amending footgun 3 and the `allowlist` module's paragraph to state that losing the `AllowlistAdmin` on a sale that has met its soft cap locks the payments already taken, the buyers' allocations, and the unallocated inventory until the window closes, rather than only preventing further purchases. Consider also bounding the window in `create_sale` with a maximum duration, so that the lock can never outlast it, weighing that against the legitimately long-dated sales such a bound would reject.

Notes & Additional Information

N-01 Event Schemas Omit Enforcing Types, Hindering Event-Only Filtering

Two events advertise configuration payloads without naming the types that enforce them. `SaleCreated` carries `CurveParams` in its type tag but not the `Curve` witness that controls pricing, and `VestingScheduleParamsSet`, emitted when a vesting policy is attached, carries `VestingScheduleParams` but not the `VestingWitness` that gates release. Both enforcing types exist only as parameters of the `PrefundedSale` object type, so a consumer filtering on event type tags cannot tell which module interprets the payload it received, and cannot subscribe to sales of one specific curve or vesting implementation without reading the sale object. That object read is the gap's true measure: the created object's type tag already names both witnesses, and the caller's own transaction states them explicitly, so the omission costs an event-only consumer a lookup rather than concealing anything on-chain.

The consequence shows most clearly in curve identification. `fixed_rate_curve::params` is public and its documentation invites external construction, so any party can declare a curve witness module of its own and create a sale over that witness paired with a genuine `fixed_rate_curve::Params`. Such a sale is fully operable, because `mint_activation_ticket` is gated on the `Curve` witness the impostor itself declares, and the impostor need not read the `Params` at all: `mint_quote` takes `rate` as a plain argument and computes the allocation from it, never reading the `curve_params` the sale stores and never interprets. The emitted `SaleCreated` is then schema-identical to an honest fixed-rate sale's, so a monitor whitelisting known-good curves from the event stream alone cannot separate the two. The deception is narrow, since `purchase` is generic over `Curve`, so the impostor's fully qualified type appears in the buyer's own type arguments rather than merely as a callee address. The vesting slot carries the same silence over an incoherent witness, a misconfiguration that strands every buyer's allocation.

Consider adding `Curve` as a phantom type parameter to `SaleCreated` and `VestingWitness` to `VestingScheduleParamsSet`, so that the emitted type tag names the module enforcing the configuration each event advertises and an event-only consumer can filter on it directly.

N-02 Misleading Error Messages

The `prefunded_sale` module raises aborts through `#[error]` constants whose message string is rendered to the caller. Several of these messages misstate the condition their guard checks, most often because one constant is shared by guards with different predicates, so the prose is accurate at one call site and wrong at another. Every affected transaction aborts correctly and atomically, so the impact is limited to diagnosability: a caller or an off-chain handler acting on the message is pointed away from the actual cause.

- The `ESaleWindowStillOpen` message states that the sale "is still open and has not sold out". This matches the compound guard in `finalize`, which passes once the window closes or the hard cap is reached, but not the window-only check in `cancel_after_close`, which consults neither `raised` nor `hard_cap`. A sold-out sale still inside its window therefore aborts there with a message falsely stating that it has not sold out and, through the word "yet", implying that waiting will help. It will not: once the window closes, the next assertion aborts with `ESoftCapMet`, since `raised >= hard_cap >= soft_cap` (the cap ordering is enforced at creation). `cancel_after_close` can therefore never succeed on that sale, while `finalize` is already legal in that exact state.
- The `ESoftCapMet` message ends with the clause "or none was set". That clause is a real abort trigger at `cancel_after_close`, whose guard is a conjunction in which `soft_cap == 0` blocks the cancel. At `cancel_emergency` the guard is a disjunction in which `soft_cap == 0` permits the cancel, so the clause names a condition under which that guard passes and tells the caller the opposite of the rule that aborted the transaction. The constant's doc comment and the per-function `Aborts` entries of `cancel_after_close` and `cancel_emergency` all state the correct per-site conditions; only the runtime string conflates the sites.
- In `pair_refund_vault`, the cap check raises `EWrongVault`, whose message reads "The provided refund vault is not the one paired with this sale". The assertion one line earlier establishes that no vault is paired yet, so the message describes a pairing that provably does not exist; the real condition, that the supplied `RefundVaultCap` does not control the supplied vault, is stated correctly in the function's `Aborts` list. The same abort code also guards the genuine paired-vault comparisons in `share_and_activate`, `finalize`, `refund`, and the internal cancel path, so an off-chain handler cannot distinguish a wrong cap at pairing time from a wrong vault later in the lifecycle.
- The `EActivationAfterClose` doc comment and message state that activation was attempted after the closing time "has passed", but the guard uses a strict inequality and also aborts at exactly `now == closes_at_ms`, an instant at which the closing time

has not passed and `purchase` is still permitted. The per-function doc already states the correct inclusive condition, `now >= closes_at_ms`.

Consider aligning each constant's message and doc comment with every guard that raises it, and splitting a constant into distinct errors where one code covers two different predicates, for example a dedicated wrong-cap error for the pairing-time check in `pair_refund_vault`.

N-03 `Quote` Does Not Guarantee Fresh Sale State Within a Transaction

A `Quote` is the hot potato that carries a priced purchase from the curve module to the sale, holding the sale id, the buyer's payment, and the curve-computed allocation. Because the struct has no abilities, it cannot be stored, transferred, or replayed, so a quote never crosses a transaction boundary and cross-transaction staleness is impossible. Within a single programmable transaction block, however, no such guarantee exists: `mint_quote` takes the sale by immutable reference, so several quotes can be minted up front, and commands between minting and consumption can mutate the sale, most directly other purchases in the same transaction block.

The `purchase` function never reprices a quote: it checks sale-id equality, validates the payment amount against the sale's caps, and accepts the carried allocation verbatim, subject only to the non-pricing inventory bound, while itself advancing `raised` and `total_allocated`. For the shipped `fixed_rate_curve` this is harmless: its `quote` function reads the rate from a `Params` value fixed at sale construction with no mutator, so a delay within the transaction block never changes the price. The hazard is confined to custom state-dependent curves, which the pricing-agnostic `Curve` generic can host. Under a curve pricing from the public `raised` or `total_allocated` views (a bonding curve, for example), a quote minted in a cheap pricing region would execute at that stale price even after an earlier purchase in the same transaction block advanced the sale into a more expensive region, creating a time-of-check to time-of-use gap between mint and consumption. The party bearing the cost is the issuer: a buyer who assembles the block executes the stale quote at the cheaper price, and since `purchase` is permissionless on the shared sale, any buyer can arrange this deliberately. No party can make another buyer's quote stale. The design commentary above the struct frames the curve as a trusted first-party component and enumerates the carrier's guarantees (single-use, non-replayable, sale-bound), but it never names freshness as a property the carrier does not provide, so a curve author reading that block has no cue that a quote may be consumed at a stale price.

Consider stating the freshness contract in the `Quote` design commentary and in the `mint_quote` documentation that curve authors build against: a quote's allocation reflects the sale state at mint time and may be consumed after later mutations in the same transaction block, so a curve pricing from mutable sale state must not depend on a quote being fresh at consumption. If a structural guarantee is preferred for future curves, an alternative is to bind quotes to a sale-state version, stamped at mint and advanced by `purchase`.

N-04 Vesting Lockup Guarantee Is Not Implied by the Documented Curve Contract

The vesting-attached sale documents schedule bypass as impossible, without qualification. The `set_vesting_schedule_params` documentation asserts as a headline that "The schedule cannot be bypassed" and then offers the witness-pinning argument as its reason, and the same unconditional claim recurs in the type documentation and in `claim_into_vesting`. Where the pinned witness is a genuine module-private curve witness, that reasoning is sound as far as it reaches, and the analysis below assumes it. The guarantee still does not follow, because it rests on a property the curve contract never states: that a deposit raises the releasable amount by no more than the deposit itself. `claim_into_vesting` hands the buyer the wallet by value, so the buyer can deposit into it, and unless the extra amount that becomes releasable is at most the amount deposited, depositing pays for itself and the schedule is defeated.

The curve contract in `openzeppelin_finance` requires no such thing. The module-level statement reads that the curve "must be monotonically non-decreasing in time and bounded above by `balance + released`", and the obligation is restated in exactly those two terms. Monotonicity is scoped to time; nothing constrains the curve's behavior with respect to the total, which is what a deposit moves, and the one place deposits against the total are discussed is permissive rather than restrictive. A threshold curve that vests nothing while the total is below some level and everything once it is reached satisfies both documented conditions, since a function constant in time is non-decreasing and the bound holds with equality at the threshold. A buyer whose allocation falls below that level deposits the difference, releases the full amount, and nets the entire allocation immediately, with the payout forced to the buyer in any case. Because the promise lives in `openzeppelin_sale` while the contract lives in `openzeppelin_finance`, and the sale holds no enforcement lever once it returns the wallet, the sale cannot close the gap on its own.

This is a hole in the specification rather than a misuse of trust: an honest curve author who satisfies every documented requirement, following the documented template, can defeat the

lockup without knowing. Two caveats bound the impact: the issuer pins the curve, not the buyer, and the shipped linear curve is safe, so no shipped configuration is affected.

Consider qualifying the guarantee at each site that states it unconditionally, so that it names the property the pinned curve must satisfy rather than asserting bypass is impossible outright.

N-05 Missing Documentation

Several documentation passages in the `sale` package omit information that integrators rely on:

- The module header of `prefunded_sale` presents its configuration model as complete, stating that "the sale supports four orthogonal configuration axes", yet none of that enumeration, the lifecycle diagram, the permissionless list, or the sale-shapes table mentions vesting. Vesting is a fifth Init-phase axis: `set_vesting_schedule_params` is a one-shot setter that irreversibly switches redemption from `claim` and `claim_all`, which then abort with `EClaimRequiresVesting`, to `claim_into_vesting` and `claim_all_into_vesting`, which are permissionless yet absent from the permissionless list. The gap is confined to these header overview sections: the struct and function documentation covers vesting, but a reader shaping a sale from the header alone has no signal that a fifth axis exists. The package `README` repeats the four-axis count.
- The `purchase` documentation presents sizing the payment to `hard_cap() - raised()` off-chain as the remedy for the all-or-nothing hard-cap rule, but omits that `raised` is read from a shared object and is stale by construction. A concurrent purchase can push a correctly sized payment over the cap, and the whole transaction then aborts with `EHardCapExceeded`. The payment is never consumed on the abort, so the cost is gas only, but no slippage or partial-fill mechanism exists to close the race. The practical remedy near sell-out is under-sizing the payment or retrying, neither of which the documentation mentions.
- Nothing documents that `EAllocationOverflow` can preempt `EHardCapExceeded`. The quote-side `u128` guard runs before the hard-cap check in `purchase`, so on a sale configured at the boundary `hard_cap == floor(u64::MAX / rate)`, which the activation guard permits, a payment whose own value exceeds `hard_cap` aborts at quote time with `EAllocationOverflow` ("The token allocation would be too large to represent") instead of `EHardCapExceeded`. The activation guard guarantees that a payment within the cap never sees the overflow error, so the consequence is only a misleading abort at an extreme configuration; a note on the `EAllocationOverflow` documentation closes the gap.

Consider adding these passages: vesting as a fifth axis in the header overview sections, the staleness caveat in the `purchase` sizing guidance, and the error-precedence note on `EAllocationOverflow`.

N-06 Code Style Inconsistencies

The sale package deviates from the style guide or from its own conventions in the following three respects:

- The two configuration events that embed a non-phantom payload type order their generic parameters oppositely. `SaleCreated` places the payload first, as `SaleCreated<CurveParams, phantom SaleCoin, phantom PaymentCoin>`, while `VestingScheduleParamsSet` places it last. The module's other twelve events are uniformly `<phantom SaleCoin, phantom PaymentCoin>`, so `SaleCoin` and `PaymentCoin` hold fixed positions in every event type tag except that of `SaleCreated`, forcing reflective tooling that resolves the sale events for a given coin pair to special-case it. Neither ordering is wrong on its own; carrying both is the inconsistency. Reordering either declaration changes that event's type tag, which any consumer decoding it would have to follow.
- The style guide requires every event to carry a `///` summary of the form "Emitted by `fn` when ...". The `Claimed` event's summary reads "Emitted for each receipt redeemed through `claim` / `claim_all` (and the vesting variants)." and is the only one in the package that drops the "Emitted by `fn`" opening. The mandated form accommodates multiple emitting functions, as `SaleCancelled` demonstrates in the same file with "Emitted by `cancel_after_close` and `cancel_emergency` when the sale is cancelled.", so a compliant phrasing that names the public redemption entry points is available.
- The "#### Aborts" list of `refund_vault::deposit` omits the native arithmetic overflow from its `balance::join` call, which the style guide requires listing, native arithmetic overflow and cross-module aborts included, marked unreachable where they are. The sibling `prefunded_sale::deposit` documents the same overflow, so the two deposit functions are inconsistent.

Consider giving the two payload-carrying events a single, consistent generic-parameter order, rewording the `Claimed` summary into the form the style guide mandates, and adding the missing native-overflow entry to `refund_vault::deposit`'s Aborts list.

N-07 External Integrators Cannot Reliably Distinguish Finalized From Cancelled Sales

An external package that routes a buyer to `claim` or `refund` on-chain must first decide whether a sale ended in `Finalized` or `Cancelled`, and the sale's own public surface cannot answer that question. For example, a sale configured with no soft cap and finalized after the window closed with `raised < hard_cap` is indistinguishable from the same sale closed through `cancel_emergency`: the two are identical on all three of the public views that bear on the decision. `is_open` returns `false` whenever the phase is not `Active`, so both read `false`. `has_reached_soft_cap` compares `raised >= soft_cap`, which any nonnegative `raised` satisfies at a zero soft cap, so both read `true`. `has_reached_hard_cap` reads `false` for both, with the cancelled side guaranteed by the assert that emergency cancellation requires `raised < hard_cap`. `proceeds_amount` differs immediately after the transition but is zeroed on both sides once the admin withdraws, so it is not a reliable signal either. The collision requires the window-close finalize; a sold-out finalize leaves `has_reached_hard_cap` true and the two phases separate again.

The cause is the visibility of the phase predicates. `phase` is public and returns a `Phase`, but the four predicates that interrogate that value are `public(package)`, and Move 2024 permits matching enum variants only inside the defining module, so an external package receives the value but cannot interrogate it. The one route that does work runs through the paired vault, whose predicates are public: `finalize` flips the vault to `Closed` and then sets `Finalized`, while `do_cancel`, the body shared by both cancel paths, flips it to `Refunding` and then sets `Cancelled`. A vault reading `is_closed` means claim and one reading `is_refunding` means refund.

The issue is that the vault route is the only one available. It forces an integrator to depend on a phase-to-vault-state coupling that is an implementation detail rather than a documented part of the sale's API: the vault's module documentation disclaims any knowledge of sales, and `finalize`'s documentation presents the flip as keeping vault state in step with the sale's terminal phase, not as a supported read path. Furthermore, the route cannot be verified. `refund_vault_id` is read only inside the module and has no public accessor, so a wrapper handed a sale and a vault cannot check that the vault is that sale's vault before trusting its state as a phase signal. A caller can supply an unrelated `RefundVault` of the right payment coin type, flipped independently, and the wrapper's inference becomes whatever the caller chose. Misrouting is nonetheless harmless on its own, because a wrong vault into `refund` aborts with `EWrongVault` and a wrong phase into claiming aborts with `ENotFinalized`, both pure aborts that change no state. The benefit of a reliable read path is limited to avoiding

a wasted transaction and making the inference trustworthy, which makes this an API completeness concern rather than a defect in the state machine.

The style guide directs contributors to keep visibility restricted and expose only the stable external API as `public`, under which the sale's package-visible predicates are the compliant choice and the vault's public ones are the outlier, so consistency alone would argue for tightening the vault rather than opening the sale. However, the integrator flow described above settles the direction: nothing public lets an external router read a sale's terminal phase from the sale itself.

Consider promoting the phase predicates to `public` and adding a `refund_vault_id` accessor so the vault route can be verified, and documenting the vault-state coupling if it is intended to be a supported read path.

N-08 Per-Buyer Cap Accounting Documented as an Accumulating Total

The prefunded sale supports an optional per-buyer payment cap, tracked per address in the `contributions` table. The tracker is a countdown: `contributions[buyer]` holds the buyer's remaining allowance, seeded at the cap on first purchase and decremented by each payment. The field documentation, the inline comment above the update, and the `EPerBuyerCapExceeded` entry in the abort list of `purchase` all describe this countdown correctly.

Three other documentation sites still describe an accumulate-upward design. The module header and the `set_per_buyer_cap` documentation both state that the cap is enforced against the "running `contributions[buyer]` total". The overflow paragraph of `purchase` goes further, claiming that `raised + paid` and `contribution + paid` are "widened to `u128` and bounds-checked before downcasting". None of that exists in `purchase`: the raised guard is a plain `u64` headroom check, the per-buyer path contains no addition at all, and the only `u128` widening in the module lives in `mint_quote`. The module therefore asserts both semantics at once. Enforcement is unaffected: `paid <= remaining` is the same predicate as `total + paid <= cap`, and oversized payments still abort with typed errors as documented; only the stated mechanism is wrong. The cost falls on readers of the prose. The sale exposes no accessor for the table, so an off-chain consumer who follows the module header and reads the raw table value through dynamic fields will interpret it as the amount contributed when it is the exact complement: a buyer who has contributed little reads as nearly the full cap, and a buyer who has exhausted the cap reads as zero. A maintainer

trusting the same prose could also "restore" the documented accumulation, reintroducing the addition the countdown eliminated.

Consider correcting the three sites to match the field documentation, stating that `contributions[buyer]` holds the remaining allowance, seeded at `per_buyer_cap` on first purchase and counted down, and rewriting the overflow paragraph of `purchase` to describe the `u64` headroom check and the countdown, reserving the `u128` language for `mint_quote`. Consider also renaming the `contributions` field (for example, to `remaining_allowance`) so the name stops suggesting a contribution total.

N-09 Incorrect Documentation

Several documentation passages in the `prefunded_sale`, `fixed_rate_curve`, `allowlist`, and `refund_vault` modules describe behavior that the implementation does not exhibit:

- The `refund_vault_id` field documentation states that the id becomes `Some` "once activated". The field is filled by `pair_refund_vault`, which asserts the `Init` phase, and `share_and_activate` dereferences it before the sale is activated, so the field is populated strictly before activation.
- The `vesting_schedule_params` field documentation and the `claim_into_vesting` documentation state that the vesting schedule is "fixed at (sale) construction". `create_sale` takes no vesting argument and always sets the field to `none`; the sole writer is `set_vesting_schedule_params` during the `Init` phase. The view documentation states this correctly, and the same phrase is used for `curve_params`, which genuinely is a `create_sale` argument, so "fixed at construction" carries two different meanings in the same file. The property the documentation asserts (that the buyer cannot influence the schedule) is true and enforced.
- Three comment sites name a `Quote` parameterization that does not exist: `Quote<C>` in `prefunded_sale` and `Quote<FixedRateCurve>` in both the module rationale and the witness documentation of `fixed_rate_curve`. `Quote` is declared with a single `PaymentCoin` type parameter and carries no curve tag; the curve binding is transitive rather than structural, since `mint_quote` stamps `sale_id` from the sale reference and `purchase` re-checks it. The correct phrasing already exists a few lines below.
- A comment in `allowlist` calls admin issuance "idempotent on the sale side". `enable_allowlist` aborts on a second call with `EAllowlistAlreadyEnabled`, and its own documentation describes it as one-shot. The package uses "idempotent"

with its precise meaning elsewhere, where a repeat call succeeds with no additional effect, so the comment asserts the opposite of the implemented behavior.

- The `cancel_emergency` summary prose states that a sale "that has met its soft cap must `finalize` instead". Per the guards, a sale with `soft_cap == 0` can be emergency-cancelled at any `raised < hard_cap`, and the `ESoftCapMet` aborts-list entry already carves out the zero case ("if a soft cap is configured"). The behavior is intended, and the guard itself must not be tightened, as it is the sole cancellation route for a sale without a soft cap; only the summary sentence misleads.
- The `Cancelled` bullet in the integrator-footguns section calls the pinned allocation "Buyer's tokens", locked "until they call `refund`". However, `refund` returns only the payment and releases the allocation back to the unallocated pool that the admin-gated `withdraw_unsold_inventory` pays out. A buyer can never obtain sale tokens from a cancelled sale, as claiming requires the `Finalized` phase.
- Two sites state that sold-out and hard-cap-reached "coincide": the module documentation and the `share_and_activate` documentation. Activation asserts `inventory >= required_inventory`, and depositing more than the required backing is a supported configuration, so a sale at its hard cap can still hold surplus inventory: hard-cap-reached does not imply sold out. The package `README` repeats the claim.
- The sale-shapes table labels the capped public round "Anti-whale public sale" with KYC "no". The per-buyer cap is keyed by the sender address in the cap accounting, so one actor buying from many addresses defeats it. A per-address cap does not deliver whale resistance without Sybil resistance, which only the KYC-gated shape provides. The `README` repeats the table row.
- Both the `purchase` aborts-list entry for `EPerEntryCapExceeded` and the error constant documentation present the per-entry cap abort as unconditional, omitting that the guard treats `max_amount == 0` as no per-entry cap. The sentinel is documented on the allowlist side, in the field and parameter documentation, but a reader of the sale-side documentation alone would conclude that a zero-cap entry blocks every purchase, the inverse of the actual behavior.
- The comment "only buyer can claim" on the beneficiary argument in `claim_all_into_vesting` does not hold for an empty receipts vector: the per-receipt buyer check runs only inside the batch loop, so any sender obtains a zero-balance `VestingWallet` naming itself beneficiary. This is harmless, since `vesting_wallet::new` is permissionless and the path grants nothing the caller could not already do directly, but the comment overstates the gate.
- The `refund_vault` state diagram and `Refunding` variant documentation state that depositors claim their funds back individually, but the vault holds no depositor ledger

and its only `Refunding`-state egress, `release_balance`, is gated on the controller cap and takes no claimant. In the paired flow the cap is sealed inside the sale, so a buyer reaches the funds only through `prefunded_sale::refund`, authorized against the `Receipt`; the per-buyer protection is the sale's, not the vault's. A reader adopting the vault standalone on the strength of these lines gets a cap-controlled escrow with no per-depositor accounting, and the package `README` repeats the claim in its module table.

- The `new` documentation prescribes the paired-sale flow as `new`, then pair, then `share`, but `share` consumes the vault by value while `share_and_activate` also requires it by value and shares it internally, so the prescribed sequence cannot be assembled into a valid transaction. Pairing does not require an unshared vault either, since `pair_refund_vault` takes it by reference. The correct flow is documented on `share_and_activate` itself, and the README's "Common mistakes" row misstates the same flow, claiming a shared vault cannot be paired and prescribing a manual share after activation.

Consider correcting each passage to match the implemented behavior. Where a correct statement already exists elsewhere in the package, aligning the inaccurate site with that phrasing is sufficient.

N-10 Load-Bearing `ENotInit` Guard Mislabeled as Purely Defensive

`share_and_activate` consumes an `Init`-phase `PrefundedSale` by value together with an `ActivationTicket`, flips the phase to `Active`, and shares the sale and its paired vault. The comment above its phase check reads: "The only way to hold a `PrefundedSale` by value is straight out of `create_sale`, so this is a purely defensive check." The premise is false. The struct carries only the `key` ability, so the sale can never return to owned status, but a transaction may take a shared object by value when the call consumes it. The other half of such a call is equally available: `mint_activation_ticket` performs no phase check, and the public `fixed_rate_curve::activation_ticket` wrapper takes the sale by reference, so a fresh ticket can be minted against a live shared sale in any phase.

What the assert uniquely protects is narrow and load-bearing. Every terminal sale whose window has closed is already covered by `EActivationAfterClose`, so the two states left to `ENotInit` are the in-window terminal transitions: a finalize on the hard-cap disjunct, and `cancel_emergency`, which is in-window by construction. For a sale finalized early on its hard cap, every other guard passes: fresh ticket, held vault cap, matching vault id, open window, untouched inventory. Only `ENotInit` separates such a call from the phase write

back to `Active`, and a sale reset that way would be permanently bricked. `finalize` would reach `flip_to_closed`, which requires an `Active` vault that is already `Closed`. The two cancel paths abort before they reach the vault, `cancel_after_close` on `ESoftCapMet` and `cancel_emergency` on `ESaleAlreadyComplete`. Redemption aborts on claiming's `Finalized` guard, so every unclaimed allocation is stranded. In the emergency-cancel variant `purchase` stays live, so new buyers could pay into a sale that can never reach any terminal phase.

None of this is reachable as shipped, because the assert rejects the re-passed sale before any state is rewritten. The comment's error is calling the assert redundant. Sui permits re-sharing an already-shared object at its original shared version, so the `transfer::share_object` reached after the phase write does not abort on a sale passed back in by value. `ENotInit` is therefore the only guard preventing the reset above, and a maintainer who deletes it on the comment's advice makes the bricked state reachable.

Consider rewriting the comment so that it carries its own justification: the check rejects a shared, non-`Init` sale passed back in by value, and is load-bearing because the module's remaining guards do not cover a terminal sale whose window is still open. Consider also adding a phase check to `mint_activation_ticket`, closing the ticket half of the call.

N-11 Inconsistent Zero-Amount Event Suppression

Three fund-moving functions in the package skip their event when the call moves no value, and each documents that behavior. `refund_vault::deposit` consumes the balance but returns before the emit when the amount is zero, and its documentation calls a zero-value deposit a no-op that emits no `VaultDeposit`. `withdraw_proceeds` and `withdraw_unsold_inventory` emit only for a positive amount, and both document the empty call as idempotent, stating that a second call returns an empty balance and emits no event.

`refund_vault::withdraw_all` is the structural twin of `withdraw_proceeds`: both authorize, assert a terminal state, read the whole balance, split it, emit, and return. It shares the idempotency as well, since `Closed` is absorbing (both flips assert an `Active` source state), so a second call passes its guard, splits zero, and returns an empty balance. Yet it emits `VaultRelease` unconditionally, hardcoding `locked_after: 0`, and its documentation records no idempotency. One module therefore holds both conventions: `deposit` suppresses at zero while its drain counterpart does not. The zero-amount emission also contradicts the `VaultRelease` documentation, which states the event is emitted "when funds leave the

vault". No funds are at risk, and the emitted fields are accurate in every case, so the cost falls only on a consumer that treats the event's presence, rather than its fields, as the signal that a vault was drained. The exposure is confined to the standalone escrow the module documentation advertises, because `withdraw_all` is unreachable in the paired flow: `finalize` leaves the proceeds in the sale and flips an empty vault to `Closed`, the cancel path routes them to `Refunding` instead, and the controller cap is sealed inside the sale at pairing, so no sale-side code ever calls it.

A smaller instance of the same divergence sits across the two deposit functions: `prefunded_sale::deposit` emits `InventoryDeposited` unconditionally where its vault-side counterpart guards, though the call is `Init`-only on a sale that is still an unshared value, so only the issuer can reach it. `release_balance` is a different case and is deliberately excluded here: its amount is caller-supplied, so a zero is a caller error rather than a repeat call, and the package's precedent for that is to reject the input, as `mint_quote` does with `EZeroPayment`.

Consider guarding `withdraw_all`'s emit at zero and documenting the idempotency its twin already documents, and aligning `prefunded_sale::deposit` with `refund_vault::deposit`. If unconditional emission is intended at either site, consider recording that intent on the function and rewording the `VaultRelease` description so that it no longer ties the event to funds leaving the vault.

N-12 Lifecycle Events Are Not Self-Contained for Off-Chain Consumers

The `prefunded_sale` and `refund_vault` modules emit an event for every lifecycle transition, and many of those events are self-contained for a stateless off-chain consumer: they carry the applied delta, the resulting post-state, or the id of a freshly created object. The surface is inconsistent, however: several functions move state, create an object, or verify a value that their event does not carry, and most such omissions have a sibling event in the same package that carries the analogous value:

- The three capability-minting sites disagree on whether the new capability's id belongs in the creation event. `enable_allowlist` emits the new `AllowlistAdmin`'s id through the `allowlist_admin_id` field, while `create_sale` constructs the `SaleAdminCap`, the object that gates emergency cancellation and the proceeds and unsold-inventory withdrawals, immediately before emitting `SaleCreated` without its id. `refund_vault::new` likewise constructs the `RefundVaultCap` on the line before the emit and omits its id from `RefundVaultCreated`. An event-only consumer

therefore cannot identify the objects that control cancellation and withdrawals, although the transaction's object-change effects do recover them.

- The `Refunded` event carries only the paid amount, while the refund path also decrements `total_allocated` by the receipt's allocation. The adjacent `Claimed` emit reports exactly the delta it applies. A consumer tracking `total_allocated` must therefore join each `Refunded` back to its originating `Purchased` through `receipt_id`, a stateful join needed nowhere else in the package.
- No event in the package carries a post-state for `total_allocated`, although the pattern exists for the other running counters: `Purchased` carries `raised_after`, `InventoryDeposited` carries `new_inventory`, and the vault's `VaultDeposit` and `VaultRelease` events carry `locked_after`. The purchase path moves `total_allocated` while reporting only the raised post-state, and `Claimed` reports the post-state of neither counter `claim_internal` moves. The convention is partial rather than broken in a single place, and completing it requires a `total_allocated_after` field on `Purchased`, `Claimed`, and `Refunded` together. Without it, a late subscriber reconstructing the counter that bounds the `withdraw_unsold_inventory` payout from the event stream alone must replay the sale's full history, though a single read of the public `total_allocated` view returns it directly.
- `share_and_activate` destructures `required_inventory` off the `ActivationTicket` and uses it once, in the check that the deposited inventory covers the curve's committed backing. The `SaleActivated` event then records only `sale_id` and `activated_at_ms`, so the one number activation exists to verify is absent from the activation event, though for the fixed-rate curve it is recomputable as `hard_cap * rate` from `SaleCreated`.

Consider emitting the capability ids in `SaleCreated` and `RefundVaultCreated`, the allocation delta on `Refunded`, a `total_allocated_after` field on the three events that move that counter, and `required_inventory` on `SaleActivated`, so that each event carries the state its emitting function moves or verifies. Every such addition changes the shape an integrator decodes.

N-13 Missing Batch Refund Counterpart to `claim_all`

The success path of the sale offers two batch redemption variants, `claim_all` and `claim_all_into_vesting`, each of which consumes a `vector<Receipt<SaleCoin>>` and redeems every receipt through the shared `claim_all_internal` loop. The failure path

has no equivalent: `refund` takes a single `Receipt<SaleCoin>` by value, so a buyer holding several receipts on a cancelled sale needs one `refund` call per receipt.

The module documentation reinforces the expectation of symmetry: the permissionless operations list names `claim_all` alongside `refund` with no batch refund counterpart, and the receipts note presents `claim_all` as the batching answer for multi-receipt holders, while `refund` appears there only in the buyer-binding assertion. The impact is limited to ergonomics and gas overhead rather than access to funds. Receipts are minted only to the purchasing sender, so fragmentation is self-inflicted or allowlist-driven. Every receipt still returns its full `paid` amount, and the individual `refund` commands can share a single programmable transaction block.

Consider adding a `refund_all` function that accepts a `vector<Receipt<SaleCoin>>` and mirrors the loop and empty-vector semantics of `claim_all_internal`, consuming every receipt and returning one summed `Balance<PaymentCoin>`, so that a multi-receipt buyer recovers from a cancelled sale in a single call, matching the convenience the success path already provides.

Conclusion

The `openzeppelin_sale` package implements a prefunded, fixed-supply token sale. An issuer deposits inventory up front, prices it through a replaceable pricing curve, and opens it to buyers for a bounded time window. A hard cap and an optional soft cap decide whether the sale succeeds, and buyer protection rests on non-transferable receipts and a paired refund vault. OpenZeppelin audited the five modules under `contracts/sale/sources`, assessing the correctness of the lifecycle state machine and its capability gating, the fund-conservation and cap accounting, the witness-gated curve interface and its single-use quote carrier, and the receipt-bound redemption and refund paths. The `openzeppelin_finance` vesting wallet that a vesting-enabled sale releases into was audited separately and was out of scope for this engagement.

No issues of critical, high, or medium severity were identified. Four low-severity issues and thirteen notes were raised. The low-severity issues are configuration and design pitfalls rather than exploitable vulnerabilities: an unbounded sale window that can strand a soft-cap-met sale once the allowlist admin is lost, two vesting misconfigurations the type system accepts that either strand every buyer's allocation or void the schedule lockup, and an integer-only pricing rate that silently constrains which prices a sale can express. Each stems from issuer misconfiguration or a configuration-time limitation, and none is reachable by an external party against a correctly configured sale. The notes concentrate in three areas: the accuracy and completeness of the in-code documentation, the self-containment of the emitted events for off-chain consumers, and the completeness of the external read and redemption surface. The core guarantees of fund conservation, the receipt binding that ties each redemption to its buyer, and the witness gate that confines pricing to the declaring curve were not found to be affected.

The codebase separates the sale mechanism, the pricing curve, and the refund escrow into distinct modules and is accompanied by a test suite exercising the in-scope code. Much of the sale's safety nonetheless rests outside the mechanism itself. The price a buyer pays is set by the curve, so a custom curve is trusted for correct pricing. The soft cap bounds the amount raised and not its source, so it protects buyers against an underfunded sale rather than against a determined issuer. The issuer configures the sale's window, caps, and vesting parameters, so a buyer's protection also depends on that configuration being sensible. Addressing the surfaced items would tighten the documentation, make the event stream self-describing for off-chain consumers, and remove the configuration pitfalls, reducing the chance that an issuer

ships a sale with a weaker guarantee than intended or that an integrator builds against a surface the library does not fully expose.

The OpenZeppelin development team for Sui is thanked for their cooperation and responsiveness throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.